

Unidad 3 Contenedores y Docker

DAMDAW_DAC03.- Reto 4: Ejercicios 1 y 2

Endika Peña Alonso

Módulo: DAM-DAW

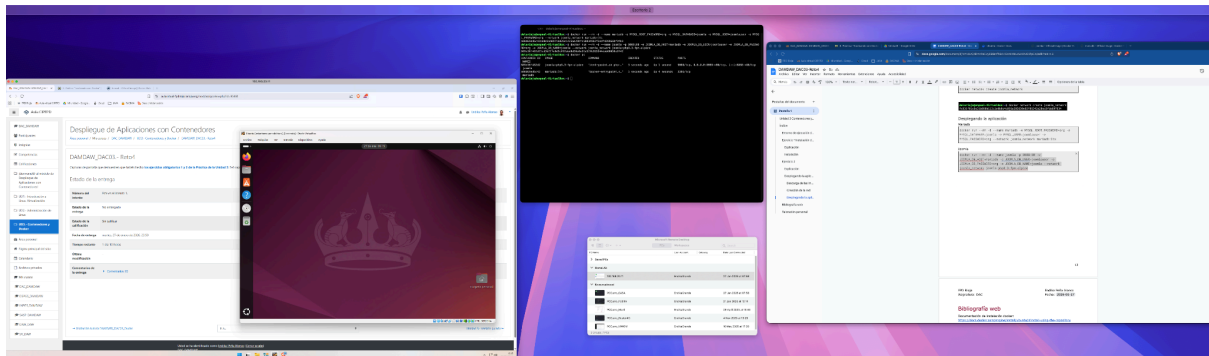
Índice

Entorno de ejecución de la práctica.....	3
Ejercicio 1 Instalación de Docker.....	4
Explicación.....	4
Instalación.....	5
Ejercicio 2.....	10
Explicación.....	10
Desplegando la aplicación.....	11
Descarga de las imágenes.....	11
Creación de la red.....	12
Desplegando la aplicación.....	12
Bibliografía web.....	13
Valoración personal.....	13

Entorno de ejecución de la práctica

El entorno de ejecución de la práctica es variado, mi equipo principal es un equipo de Apple con macOS y por compatibilidad de las prácticas me conecto a un equipo windows mediante RDP en la misma red local el cual no tiene monitor.

Hay cierto programas como VSCode o terminal que las empleo desde mi equipo apple directamente contra la máquina virtual creada con el hipervisor VBOX en mi servidor de pruebas Windows, por ello puede que durante la práctica aparezcan ciertas capturas de pantalla con interfaz de macOS y otras de windows + la VM.



Ejercicio 1 Instalación de Docker

Ejercicio 1 - Instalación de Docker

- Accede a una terminal de Ubuntu directamente o por ssh.
- Instala Docker Engine.
- Verifica que el servicio de docker corre correctamente y obtén la versión instalada.

Explicación

Vamos a instalar docker que hoy se compone de varias piezas docker-ce-cli, docker-ce, containerd.io, docker-buildx-plugin, docker-compose-plugin

Estos programas hacen distintas cosas explicación breve:

- docker-ce: Es el daemon de docker y el que levanta el socket en el servidor para realizar acciones, existen 2 versiones ce (community edition) y ee (enterprise edition)
- docker-ce-cli: Es la línea de comandos que nos sirve para interactuar con "docker"
- containerd.io: Es el runtime el que realmente ejecuta, descarga, establece las redes y los volúmenes en el S.O.
- docker-buildx-plugin: Permite entre otras cosas poder construir las imágenes para ejecutarse en distintas plataformas con arquitecturas de microprocesador distintas.
- docker-compose-plugin: Permite ejecutar acciones docker empleando un fichero de configuración .yaml anteriormente llamado docker-compose.yaml hoy en día el estándar es composefile.yaml

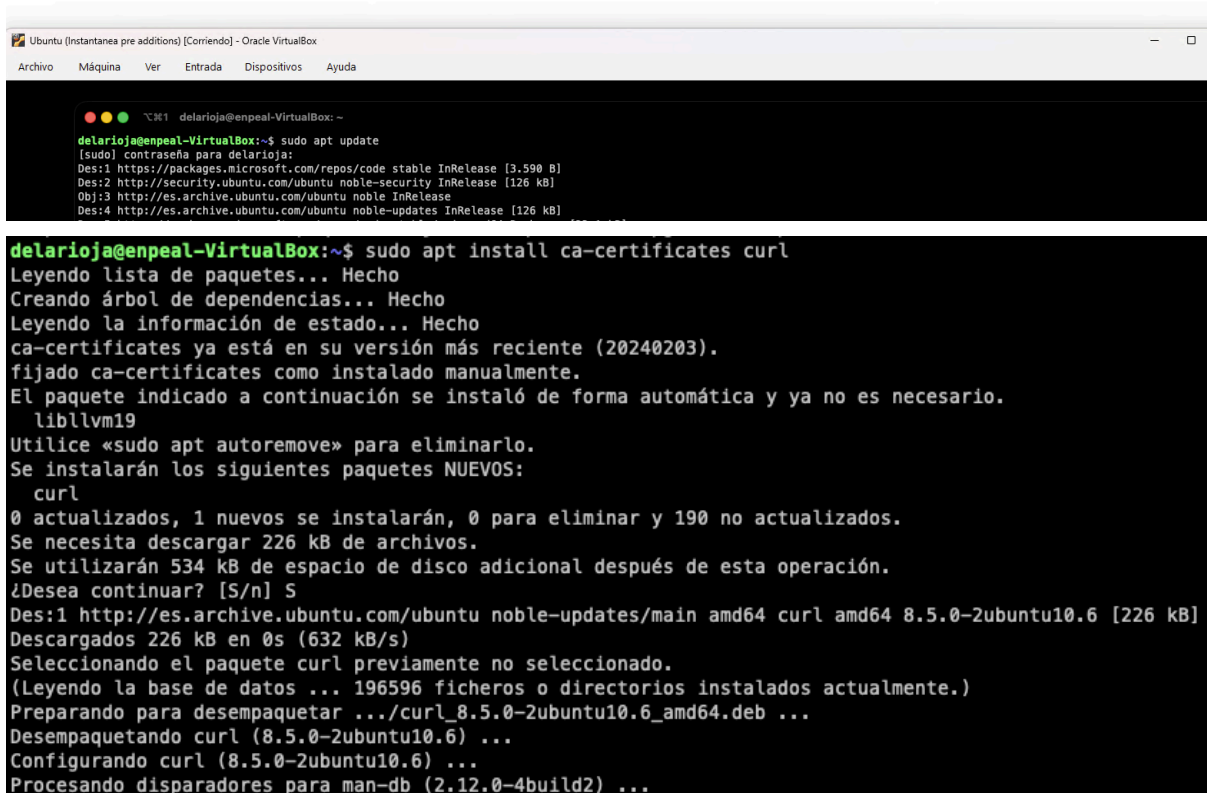
Instalación

Para la instalación se sigue la guía oficial en <https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>

1. Primer paso instalar ca-certificates y curl para poder descargar los software y poder "validar" los certificados.

```
sudo apt update  
sudo apt install ca-certificates curl
```

Endika Peña Alonso



```
Ubuntu (Instantanea pre additions) [Corriendo] - Oracle VirtualBox  
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda  
delarioja@enpeal-VirtualBox: ~  
delarioja@enpeal-VirtualBox:~$ sudo apt update  
[sudo] contraseña para delarioja:  
Des:1 https://packages.microsoft.com/repos/code stable InRelease [3.590 B]  
Des:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Obj:3 http://es.archive.ubuntu.com/ubuntu noble InRelease  
Des:4 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
delarioja@enpeal-VirtualBox:~$ sudo apt install ca-certificates curl  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias... Hecho  
Leyendo la información de estado... Hecho  
ca-certificates ya está en su versión más reciente (20240203).  
fijado ca-certificates como instalado manualmente.  
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.  
libllvm19  
Utilice «sudo apt autoremove» para eliminarlo.  
Se instalarán los siguientes paquetes NUEVOS:  
curl  
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 190 no actualizados.  
Se necesita descargar 226 kB de archivos.  
Se utilizarán 534 kB de espacio de disco adicional después de esta operación.  
¿Desea continuar? [S/n] S  
Des:1 http://es.archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.6 [226 kB]  
Descargados 226 kB en 0s (632 kB/s)  
Seleccionando el paquete curl previamente no seleccionado.  
(Leyendo la base de datos ... 196596 ficheros o directorios instalados actualmente.)  
Preparando para desempaquetar .../curl_8.5.0-2ubuntu10.6_amd64.deb ...  
Desempaquetando curl (8.5.0-2ubuntu10.6) ...  
Configurando curl (8.5.0-2ubuntu10.6) ...  
Procesando disparadores para man-db (2.12.0-4build2) ...
```

2. Importar las keys para poder descargar el software de los repositorios de docker de forma confiable.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
delarioja@enpeal-VirtualBox:~$ sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

3. Añadimos el repositorio.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

```
delarioja@enpeal-VirtualBox:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
delarioja@enpeal-VirtualBox:~$ ls -l /etc/apt/sources.list.d
total 16
-rw-r--r-- 1 root root 131 ene 26 11:02 docker.sources
-rw-r--r-- 1 root root 386 oct 24 20:06 ubuntu.sources
-rw-r--r-- 1 root root 2552 ago 5 18:54 ubuntu.sources.curtin.orig
-rw-r--r-- 1 root root 278 nov 14 22:55 vscode.sources
delarioja@enpeal-VirtualBox:~$ cat /etc/apt/sources.list.d/docker.sources
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
```

4. Actualizamos las fuentes para que sepamos los paquetes disponibles usando los nuevos repositorios.

```
sudo apt update
```

```
delarioja@enpeal-VirtualBox:~$ sudo apt update
Obj:1 https://packages.microsoft.com/repos/code stable InRelease
Des:2 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Obj:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:5 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [42,6 kB]
Obj:7 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Descargados 91,1 kB en 1s (89,9 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 190 paquetes. Ejecute «apt list --upgradable» para verlos.
```

5. Instalamos docker.

```
sudo apt install docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin
```

```
delarioja@enpeal-VirtualBox:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias... Hecho  
Leyendo la información de estado... Hecho  
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.  
  liblvm2  
Utilice «sudo apt autoremove» para eliminarlo.  
Se instalarán los siguientes paquetes adicionales:  
  docker-ce-rootless-extras git git-man liberror-perl libslirp0 pigz slirp4netns  
Paquetes sugeridos:  
  cgroupfs-mount | cgroup-lite docker-model-plugin git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn  
Se instalarán los siguientes paquetes NUEVOS:  
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin git git-man liberror-perl libslirp0 pigz slirp4netns  
0 actualizados, 12 nuevos se instalarán, 0 para eliminar y 190 no actualizados.  
Se necesita descargar 96,1 MB de archivos.  
Se utilizarán 388 MB de espacio de disco adicional después de esta operación.  
¿Desea continuar? [S/n]
```

6. Revisamos que el daemon de docker está funcionando.

```
delarioja@enpeal-VirtualBox:~$ sudo systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)  
   Active: active (running) since Mon 2026-01-26 11:06:07 CET; 1min 9s ago  
 TriggeredBy: ● docker.socket  
    Docs: https://docs.docker.com  
   Main PID: 6966 (dockerd)  
     Tasks: 9  
    Memory: 23.1M (peak: 24.3M)  
       CPU: 721ms  
   CGroup: /system.slice/docker.service  
           └─6966 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock  
  
ene 26 11:06:06 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:06.774936325+01:00" level=info msg="Restoring containers: start."  
ene 26 11:06:06 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:06.843451704+01:00" level=info msg="Deleting nftables IPv4 rules" error="exit status 1"  
ene 26 11:06:06 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:06.854985644+01:00" level=info msg="Deleting nftables IPv6 rules" error="exit status 1"  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.500047694+01:00" level=info msg="Loading containers: done."  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.521196473+01:00" level=info msg="Docker daemon" commit=3b01d64 containerd-snapshotter=true storage-  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.521584493+01:00" level=info msg="Initializing buildkit"  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.567702542+01:00" level=info msg="Completed buildkit initialization"  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.572944464+01:00" level=info msg="Daemon has completed initialization"  
ene 26 11:06:07 enpeal-VirtualBox dockerd[6966]: time="2026-01-26T11:06:07.573285025+01:00" level=info msg="API listen on /run/docker.sock"  
ene 26 11:06:07 enpeal-VirtualBox systemd[1]: Started docker.service - Docker Application Container Engine.  
lines 1-22/22 (END)
```


7. Revisando la versión instalada

```
delarioja@enpeal-VirtualBox:~$ sudo docker version
Client: Docker Engine - Community
 Version:           29.1.5
 API version:       1.52
 Go version:        go1.25.6
 Git commit:        0e6fee6
 Built:             Fri Jan 16 12:48:47 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:           29.1.5
  API version:       1.52 (minimum version 1.44)
  Go version:        go1.25.6
  Git commit:        3b01d64
  Built:             Fri Jan 16 12:48:47 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.2.1
  GitCommit:         dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:           1.3.4
  GitCommit:         v1.3.4-0-gd6d73eb8
 docker-init:
  Version:           0.19.0
  GitCommit:         de40ad0
```

8. Inclusión del usuario delarioja en el grupo docker.

Este paso es para no tener que elevar privilegios y que el usuario pueda ejecutar el comando docker.

```
sudo usermod -aG docker "delarioja"
```

```
delarioja@enpeal-VirtualBox:~$ sudo usermod -aG docker "delarioja"
[sudo] contraseña para delarioja:
delarioja@enpeal-VirtualBox:~$ id delarioja
uid=1000(delarioja) gid=1000(delarioja) grupos=1000(delarioja),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),100(users),114(lpadmin),982(docker)
```

Ejercicio 2

Ejercicio 2 - Obtención de imágenes y generación de contenedores

- Busca en el repositorio de DockerHub la imagen oficial más reciente de JOOMLA (es un CMS similar a WordPress) y de MARIADB.
- Descarga en tu host las 2 imágenes encontradas; puedes usar la última versión con el tag "latest" aunque en producción, lo recomendable sería elegir una versión concreta.
- Crea una red para que los contenedores puedan comunicarse, con el comando: `docker network create joomla-network`
- Genera 2 contenedores en segundo plano que te permitan visualizar en el navegador web el punto de entrada de JOOMLA en el puerto 8088. Para ello lanza primero el de MARIADB, por ejemplo:

```
docker run --rm -d --name mariadb \  
-e MYSQL_ROOT_PASSWORD=org \  
-e MYSQL_DATABASE=joomla \  
-e MYSQL_USER=joomlauser \  
-e MYSQL_PASSWORD=org \  
--network joomla-network \  
mariadb:latest
```

Y luego el de JOOMLA, siguiendo el ejemplo:

```
docker run --rm -d --name joomla \  
-p 8088:80 \  
-e JOOMLA_DB_HOST=mariadb \  
-e JOOMLA_DB_USER=joomlauser \  
-e JOOMLA_DB_PASSWORD=org \  
-e JOOMLA_DB_NAME=joomla \  
--network joomla-network \  
joomla:latest
```

- Verifica que las imágenes y los contenedores son reconocidos por el demonio de Docker.

Explicación

Vamos a crear la estructura típica de la mayoría de aplicaciones en internet una aplicación y su base de datos asociada, en este contexto vamos a descargar las imágenes de joomla y mariadb.

1. Descargar las imágenes `docker pull <imagen>`
2. Crear red `docker network create <nombre_red>`
3. Lanzar los containers asociados a la red `docker run <opciones>`

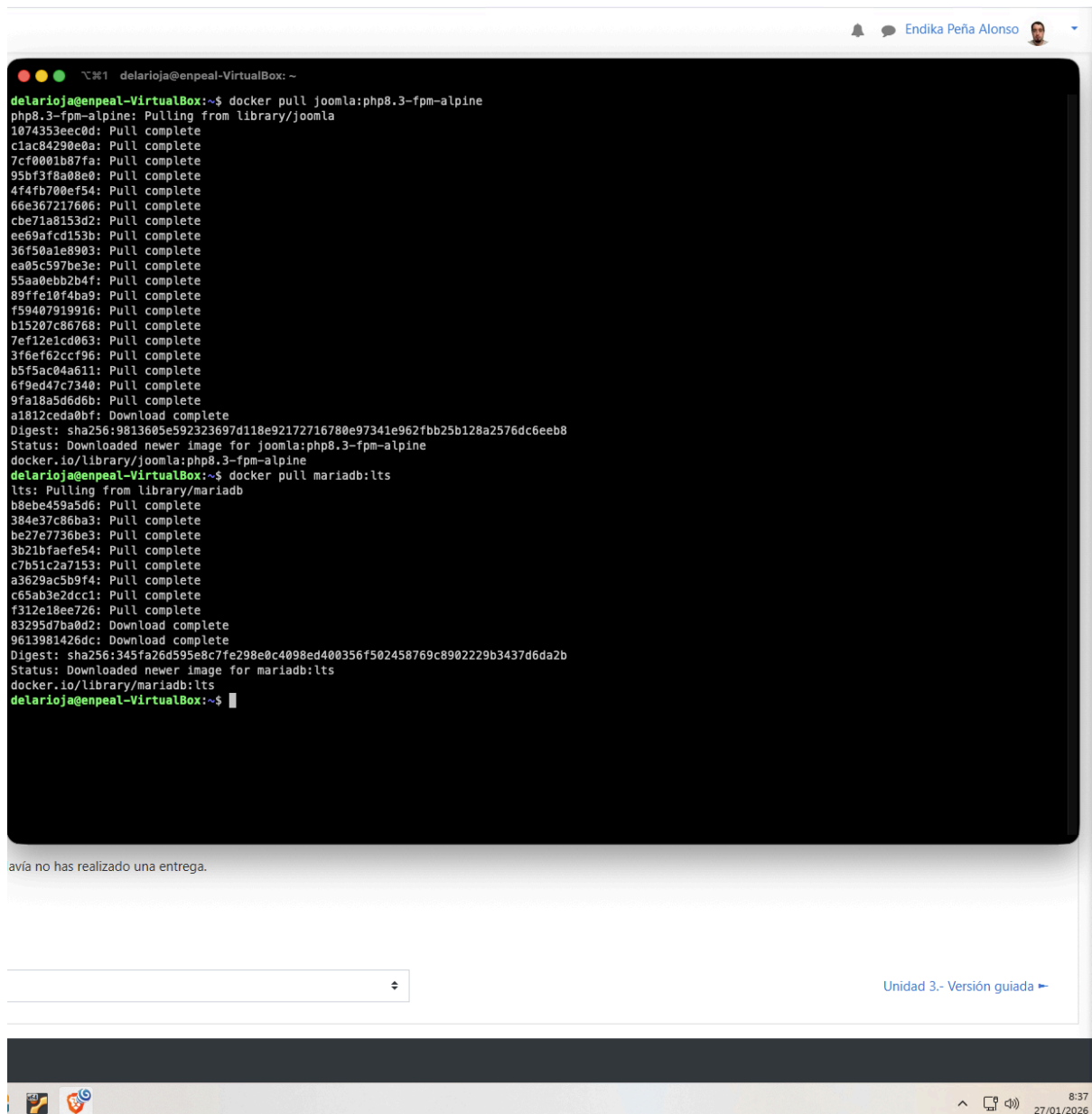
Desplegando la aplicación

Lo primero que vamos a realizar es descargar las imágenes necesarias de mariadb y joomla.

Enlaces del registro en dockerhub, es el registry por defecto en docker.

Descarga de las imágenes

```
docker pull joomla:php8.3-fpm-alpine
docker pull mariadb:1ts
```



Creación de la red

```
docker network create joomla_network
```

```
delarioja@enpeal-VirtualBox:~$ docker network create joomla_network  
fc931f01c6e31b898b112c1e0b0c4c091a285650e85f06342a28ec9fda697634
```

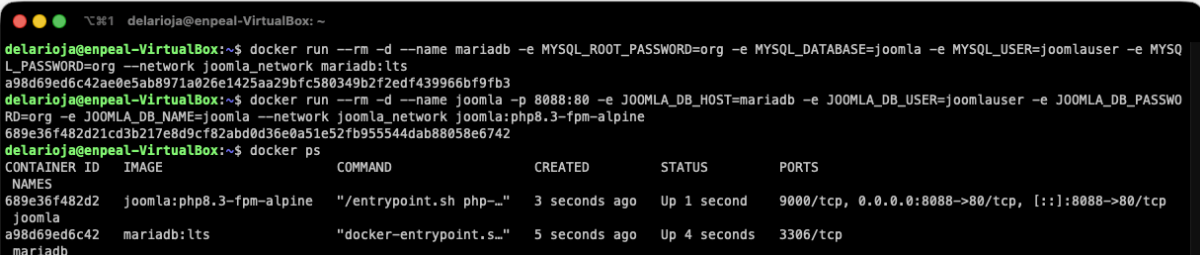
Desplegando la aplicación

Mariadb

```
docker run --rm -d --name mariadb -e MYSQL_ROOT_PASSWORD=org -e  
MYSQL_DATABASE=joomla -e MYSQL_USER=joomlauser -e  
MYSQL_PASSWORD=org --network joomla_network mariadb:lts
```

Joomla

```
docker run --rm -d --name joomla -p 8088:80 -e  
JOOMLA_DB_HOST=mariadb -e JOOMLA_DB_USER=joomlauser -e  
JOOMLA_DB_PASSWORD=org -e JOOMLA_DB_NAME=joomla --network  
joomla_network joomla:php8.3-fpm-alpine
```



The screenshot shows a terminal window with the following content:

```
delarioja@enpeal-VirtualBox: ~  
delarioja@enpeal-VirtualBox:~$ docker run --rm -d --name mariadb -e MYSQL_ROOT_PASSWORD=org -e MYSQL_DATABASE=joomla -e MYSQL_USER=joomlauser -e MYSQL_PASSWORD=org --network joomla_network mariadb:lts  
a98d69ed6c42ae0e5ab8971a026e1425aa29bfc580349b2f2edf439966bf9fb3  
delarioja@enpeal-VirtualBox:~$ docker run --rm -d --name joomla -p 8088:80 -e JOOMLA_DB_HOST=mariadb -e JOOMLA_DB_USER=joomlauser -e JOOMLA_DB_PASSWORD=org -e JOOMLA_DB_NAME=joomla --network joomla_network joomla:php8.3-fpm-alpine  
689e36f482d21cd3b217e8d9cf82abd0d36e0a51e52fb955544dab88058e6742  
delarioja@enpeal-VirtualBox:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
689e36f482d2	joomla:php8.3-fpm-alpine	"/entrypoint.sh php-..."	3 seconds ago	Up 1 second	9000/tcp, 0.0.0.0:8088->80/tcp, [::]:8088->80/tcp
a98d69ed6c42	mariadb:lts	"docker-entrypoint.s..."	5 seconds ago	Up 4 seconds	3306/tcp

Bibliografía web

Documentación de instalación docker:

<https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>

Documentación de la imagen joomla: https://hub.docker.com/_/joomla

Documentación de la imagen mariadb: https://hub.docker.com/_/mariadb

Basado en la experiencia personal y conocimientos (portfolio:

<https://endikapenia.es/>)

Valoración personal

Personalmente el reto no me aporta mucho puesto que es mi día a día en el trabajo.

Indicar que actualmente rara vez ejecuto los comandos docker de forma directa y que mayoritariamente se ejecutan empleando los composefile.yaml